Advanced SQL Queries Using Joins and Subqueries

Multi-table hotel reports with joins, subqueries, grouping, and a CTE

Prepared by: Hemanth Krishna S

Project: Hotel Management System. Output convention: result tables are expected outputs from the fixed rows in the shared SQL script; they are not screenshots of MySQL execution.

1. Active booking report using joins

This query joins bookings to guests, rooms, and hotels to show active and confirmed stays.

```
SELECT b.booking_id, g.full_name, r.room_number, h.hotel_name,
       b.check_in, b.check_out, b.booking_status
FROM bookings b
JOIN guests g ON g.guest_id=b.guest_id
JOIN rooms r ON r.room_id=b.room_id
JOIN hotels h ON h.hotel_id=r.hotel_id
WHERE b.booking_status IN ('CONFIRMED','CHECKED_IN')
ORDER BY b.check_in, b.booking_id;
```

booking_id	full_name	room_number	hotel_name	check_in	check_out	booking_status
1001	Ananya Rao	101	Lakeview Residency	2026-10-20	2026-10-24	CHECKED_IN
1002	Rahul Nair	201	Lakeview Residency	2026-10-22	2026-10-26	CHECKED_IN
1005	Arjun Das	401	Coastal Comfort	2026-10-25	2026-10-28	CONFIRMED

2. Guests with no booking history using NOT EXISTS

```
SELECT g.guest_id, g.full_name
FROM guests g
WHERE NOT EXISTS (
  SELECT 1 FROM bookings b WHERE b.guest_id=g.guest_id
)
ORDER BY g.full_name;
```

guest_id	full_name
5	Kavya Menon

NOT EXISTS is a correlated subquery: a guest appears only when no booking row matches that guest.

3. Guests with at least two bookings

```
SELECT g.full_name, COUNT(b.booking_id) AS booking_count
FROM guests g JOIN bookings b ON b.guest_id=g.guest_id
GROUP BY g.guest_id, g.full_name
HAVING COUNT(b.booking_id) >= 2
ORDER BY booking_count DESC, g.full_name;
```

full_name	booking_count
Ananya Rao	2
Rahul Nair	2

4. Guests above the average booking count using a CTE

```
WITH guest_totals AS (
  SELECT g.guest_id, g.full_name, COUNT(b.booking_id) AS booking_count
  FROM guests g LEFT JOIN bookings b ON b.guest_id=g.guest_id
```

```
  GROUP BY g.guest_id, g.full_name
)
SELECT full_name, booking_count
FROM guest_totals
WHERE booking_count > (SELECT AVG(booking_count) FROM guest_totals)
ORDER BY booking_count DESC, full_name;
```

full_name	booking_count
Ananya Rao	2
Rahul Nair	2

The CTE includes every guest using LEFT JOIN, including Kavya with zero bookings. There are six bookings across five guests, so the average is 1.2; Ananya and Rahul are above the average.

5. Pending payment report

```
SELECT p.payment_id, g.full_name, p.amount,
       p.payment_method, p.payment_status
FROM payments p
JOIN bookings b ON b.booking_id=p.booking_id
JOIN guests g ON g.guest_id=b.guest_id
WHERE p.payment_status='PENDING'
ORDER BY p.payment_date, p.payment_id;
```

payment_id	full_name	amount	payment_method	payment_status
5004	Ananya Rao	2500.00	UPI	PENDING

6. Screenshot guidance

Run each report query after loading the shared seed data. Capture the query and Result Grid in MySQL Workbench. The output tables are deterministic expected results; they are not screenshots of actual execution.